ShopPilot AI - Assignment 3: SOAP Partner Integration

Design-time contract for the MsgBridge SMS gateway edge

TeamID	Roll No.	Name
22	20251651098	Uday Kumar Gupta
	20251651046	Ritesh Jadhav
	20251651059	Mukesh Mourya
	20251651052	Koushik
	20251651105	Yash

Files in the zip: integration.pdf, partner.wsdl, soap-request.xml, soap-response.xml, soap-fault.xml.

1. Context

Partner: MsgBridge SMS Gateway (an SMS gateway). **Operation integrated:** SendSms. ShopPilot AI uses it for price-drop alerts: when the Price Agent / Deal Agent says WAIT, the user can set an alert (for example, Laptop A below INR 70,000) and we text them a confirmation now and a notification when the price drops.

Why SOAP here, REST elsewhere. The rest of ShopPilot AI (products, search, recommendations, AI chat) is REST/GraphQL because those are flexible, cacheable resources that our own frontend consumes. The SMS edge is a regulated telecom integration: MsgBridge publishes a strict WSDL/XSD contract in which sender ID, recipient and the registered template ID are typed, mandatory fields, so malformed messages are rejected at the schema level and both sides can generate typed clients. SOAP also gives message-level security (account credentials travel in the envelope Header, not only in the transport) and a standard, typed fault for rejections such as an unreachable recipient. Because every message is billable and must be auditable and reconcilable (clientRef to messageId), a formal contract with explicit faults suits this edge better than a loose JSON API.

2-3. Contract and messages (separate files)

4. HTTP binding

The request in soap-request.xml travels as one HTTP POST. Method, SOAPAction and URL come directly from the WSDL binding and service/port.

POST /soap/v1/sms HTTP/1.1

Host: api.msgbridge.example

Content-Type: text/xml; charset=utf-8

SOAPAction: "http://sms.msgbridge.example/wsdl/v1/SendSms"

```
PS D:\CS543\ASSIGNMENT_1\Shop_pilot_master_branch\shop-pilot-ai> cd .\Assignment_3\
PS D:\CS543\ASSIGNMENT_1\Shop_pilot_master_branch\shop-pilot-ai\Assignment_3> python -c "import xml.etree.ElementTree as E; from pathlib import Path; fs=[Path(
'partner.wsdl'),Path('soap-request.xml'),Path('soap-response.xml'),Path('soap-fault.xml')]; [(print(f'{f}: VALID XML') if E.parse(f) else None) for f in fs]"
partner.wsdl: VALID XML
soap-request.xml: VALID XML
soap-response.xml: VALID XML
soap-fault.xml: VALID XML
PS D:\CS543\ASSIGNMENT_1\Shop_pilot_master_branch\shop-pilot-ai\Assignment_3> python -c "import xml.etree.ElementTree as E; r=E.parse('partner.wsdl').getroot()
; n={'w':'http://schemas.xmlsoap.org/wsdl/'}; print('Service:', [x.get('name') for x in r.findall('w:service',n)]); print('Port types:', [x.get('name') for x i
n r.findall('w:portType',n)]); print('Bindings:', [x.get('name') for x in r.findall('w:binding',n)]); print('Operations:', [x.get('name') for x in r.findall('.
//w:portType/w:operation',n)])"
Service: ['MsgBridgeSmsService']
Port types: ['SmsPortType']
Bindings: ['SmsSoapBinding']
Operations: ['SendSms']
PS D:\CS543\ASSIGNMENT_1\Shop_pilot_master_branch\shop-pilot-ai\Assignment_3> Get-ChildItem partner.wsdl, soap-request.xml, soap-response.xml, soap-fault.xml


    Directory: D:\CS543\ASSIGNMENT_1\Shop_pilot_master_branch\shop-pilot-ai\Assignment_3


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        13-10-2026  04:35 PM           4388 partner.wsdl
-a----        13-10-2026  04:35 PM            851 soap-request.xml
-a----        13-10-2026  04:35 PM            515 soap-response.xml
-a----        13-10-2026  04:35 PM            725 soap-fault.xml
```

HTTP request body: The complete contents of soap-request.xml, including the XML declaration, SOAP Envelope, PartnerAuth Header, and SendSmsRequest Body.

Item	Value	Taken from
Endpoint URL	https://api.msgbridge.example/soap/v1/sms	service/port soap:address
Method	POST (SOAP 1.1 always uses POST)	soap:binding transport = HTTP
Content-Type	text/xml; charset=utf-8	SOAP 1.1
SOAPAction	http://sms.msgbridge.example/wsdl/v1/SendSms	soap:operation soapAction

A success returns **HTTP 200** with soap-response.xml (status ACCEPTED means the gateway took the message, not that the phone received it). A fault returns **HTTP 500** (the SOAP 1.1 rule for any soap:Fault) with soap-fault.xml, so the consumer must read the Fault body, not just the status code.




```
PS D:\CS543\ASSIGNMENT_1\Shop_pilot_master_branch\shop-pilot-ai\Assignment_3>
>> @'
>> import xml.etree.ElementTree as ET
>>
>> SOAP = "http://schemas.xmlsoap.org/soap/envelope/"
>> MB = "http://sms.msgbridge.example/schema/v1"
>>
>> # Check successful response.
>> root = ET.parse("soap-response.xml").getroot()
>> body = root.find(f"{{{SOAP}}}Body")
>> response = body.find(f"{{{MB}}}SendSmsResponse") if body is not None else None
>>
>> assert response is not None, "SendSmsResponse is missing"
>>
>> for field in ["messageId", "clientRef", "status", "segments", "acceptedAt"]:
>>     assert response.find(f"{{{MB}}}{field}") is not None, f"Missing {field}"
>>
>> assert response.findtext(f"{{{MB}}}status") == "ACCEPTED"
>> print("Success response structure: PASS")
>>
>> # Check SOAP fault.
>> root = ET.parse("soap-fault.xml").getroot()
>> body = root.find(f"{{{SOAP}}}Body")
>> fault = body.find(f"{{{SOAP}}}Fault") if body is not None else None
>>
>> assert fault is not None, "SOAP Fault is missing"
>> detail = fault.find("detail")
>> error = detail.find(f"{{{MB}}}SmsFault") if detail is not None else None
>>
>> assert error is not None, "SmsFault detail is missing"
>> assert error.findtext(f"{{{MB}}}code") == "invalid_recipient"
>> assert error.findtext(f"{{{MB}}}reason") == "DND_REGISTERED"
>> assert error.findtext(f"{{{MB}}}retryable") == "false"
>>
>> print("SOAP fault structure: PASS")
>> '@ | Set-Content -Encoding utf8 test_examples.py
>>
>> python test_examples.py
>>
Success response structure: PASS
SOAP fault structure: PASS
PS D:\CS543\ASSIGNMENT_1\Shop_pilot_master_branch\shop-pilot-ai\Assignment_3>
```

5. Discovery

How ShopPilot AI obtains the WSDL. The primary route is a direct URL: MsgBridge publishes the contract at https://api.msgbridge.example/soap/v1/sms?wsdl. At design time the

Backend/API owner downloads it, pins it in the repo, and generates the typed client, so production does not depend on a live fetch. As a lookup, the team keeps a small **partner catalogue** (a version-controlled registry file read by the notification service), which is the modern replacement for UDDI. A new partner or version is found by looking up the service name in the catalogue, following the tModel-style pointer to the WSDL, and re-running client generation. No UDDI server is run.

Registry entry (one record):

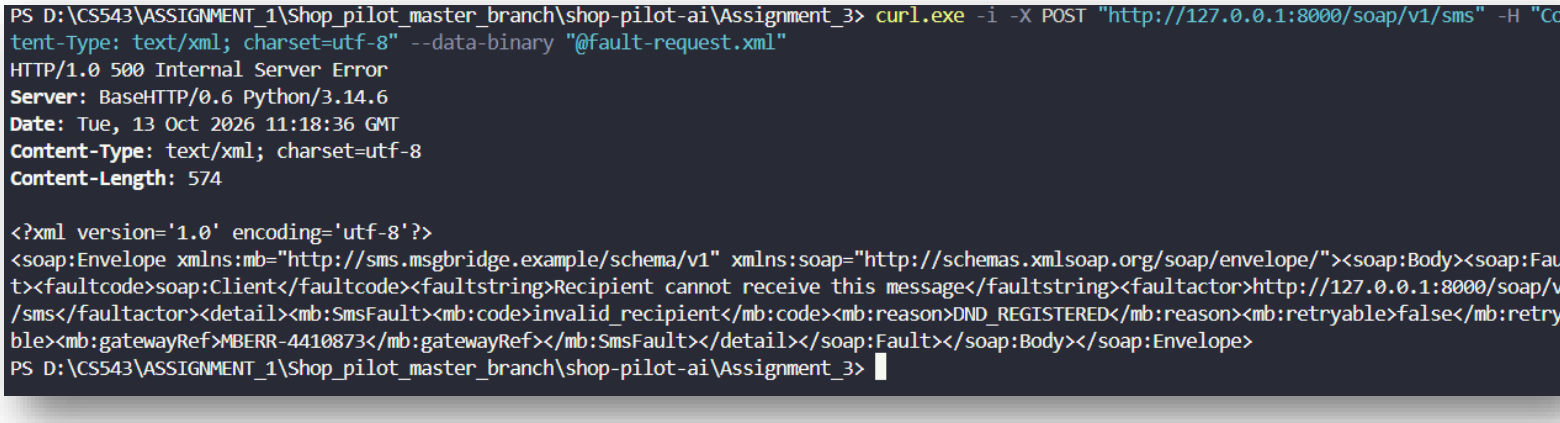

```
{
  "businessEntity": {
    "businessKey": "msgbridge-telecom-in",
    "name": "MsgBridge Communications Pvt. Ltd.", "contact": "integrations@msgbridge.example"
  },
  "businessService": {
    "serviceKey": "msgbridge-sms-service", "name": "MsgBridgeSmsService", "category": "sms-gateway",
    "description": "Transactional SMS for ShopPilot AI price alerts", "consumedBy": "shoppilot-notification-service"
  },
  "bindingTemplate": {
    "bindingKey": "msgbridge-sms-soap11",
    "accessPoint": {
      "useType": "endPoint",
      "value": "https://api.msgbridge.example/soap/v1/sms"
    },
    "protocol": "SOAP 1.1 / HTTP, document-literal",
    "soapAction": "http://sms.msgbridge.example/wsdl/v1/SendSms",
    "tModelInstanceInfo": {
      "tModelKey": "uddi:msgbridge.example:wsdl:sms:v1",
      "overviewDoc": "https://api.msgbridge.example/soap/v1/sms?wsdl",
```

```
        "localCopy": "contracts/partner.wsdl",
        "targetNamespace": "http://sms.msgbridge.example/wsdl/v1"
}
},
    "version":  "v1",
    "status": "active"
  }
```


6. Fault mapping

Assumption: the Assignment 2 contract for POST /v1/price-alerts (createPriceAlert, which sends a confirmation SMS) returns errors in the body including **422 PHONE_NOT_REACHABLE** and **503 SMS_UNAVAILABLE**.

Example:



below to match your own Assignment 2 table. An adapter in the notification service translates partner faults, so MsgBridge vocabulary never reaches a student.

MsgBridge fault (SOAP)	Adapter decision	ShopPilot createPriceAlert response
soap:Client, code invalid_recipient (any reason, e.g. DND_REGISTERED)	Not retryable. Alert saved with channel SMS marked undeliverable.	422, PHONE_NOT_REACHABLE, 'We cannot send SMS to this number. Use a different number or choose in-app or email alerts.'
soap:Server, code service_unavailable / timeout	Retry with backoff, then circuit breaker opens. Alert is saved and queued.	503, SMS_UNAVAILABLE, 'SMS is temporarily unavailable. Your alert is saved and we will retry.' (+ Retry-After)
soap:Client, code invalid_credentials / template_mismatch / insufficient_credits	Our configuration or billing problem. Alert the team, do not show detail.	502, NOTIFICATION_ERROR, 'We could not send your notification right now.'

Server Should Look like this:

